Экзамен: взаимодействие процессов

Индивидуальное задание № 3. Вариант 17

«Архитектура компьютера и операционные системы»
НИУ ВШЭ, факультет компьютерных наук

Исполнитель: Бадретдинов Азат Рустэмович
Группа: БПИ252

Москва, 2026

Сценарий и роли

В письменном экзамене каждый допущенный студент получает уникальный билет, готовит ответ, передаёт работу и ожидает своей оценки. Преподаватель проверяет одну работу за раз. Если билетов меньше, чем студентов, допускаются участники с первыми номерами в пределах числа билетов. При полной очереди готовый ответ ждёт свободного места.

Каждый студент и каждый преподаватель работает в собственном процессе. Координатор выдаёт билеты, ведёт очередь, назначает работы свободным преподавателям и печатает журнал. Для S студентов и P преподавателей в полной сессии участвуют S + P + 1 процессов. PID участников видны в журнале.

Протокол обмена

1. Участник резервирует свой номер и отправляет `HELLO`. Студент получает `PREPARE` с билетом и длительностью подготовки либо `REJECT`, если билет не выделен.
2. По окончании подготовки студент посылает `PREPARED`. Координатор фиксирует готовность и принимает работу в ограниченную FIFO-очередь при наличии места. При совпадении времени порядок зависит от поступления событий.
3. Координатор извлекает работу для свободного преподавателя и посылает `CHECK`. Преподаватель самостоятельно ждёт время проверки и возвращает `GRADE`. Два преподавателя могут проверять работы одновременно.
4. Координатор посылает оценку владельцу в `RESULT`. Студент подтверждает получение через `ACK` и завершает процесс. Недопущенный участник также подтверждает ответ и выходит.

Только координатор меняет очередь, состояния и статистику. У каждого участника свой входящий и исходящий почтовый ящик. До получения ответа участник не отправляет следующее сообщение того же обмена. Поэтому достаточно одного места на направление; данные публикуются постом семафора. При фиксированном `seed` повторяются билеты, длительности и правила оценки, а время событий и выбор свободного преподавателя могут различаться.

Реализации по уровням требований

Запуск	Организация процессов и данных
`exam-unnamed`	Базовая реализация: координатор вызывает `fork`. Сообщения и неименованные семафоры находятся в POSIX-разделяемой памяти. Семафоры инициализируются с `pshared=1`.
`exam-named`	Вторая реализация для уровня 6: участники также создаются через `fork`. Сообщения остаются в разделяемой памяти; семафоры создаются по именам через `sem_open`.
`exam-shm`	Дополнение для 7–8: координатор и программы студентов и преподавателей запускаются независимо. Участники открывают общую память и именованные семафоры по названию сессии.
`exam-mq`	Дополнение для уровня выше 8: независимый запуск, именованные семафоры и POSIX-очереди сообщений вместо общей памяти для данных. Сообщения передаются через `mq_send` и `mq_receive`.

Разделяемый объект создаётся через `shm_open`, задаётся размер через `ftruncate` и отображается с помощью `mmap`. Имена объектов содержат UID, вид транспорта и сессию. Координатор создаёт объекты исключительно, с доступом для своего пользователя, и публикует сигнал готовности после их настройки. Второй координатор с тем же именем и участник с уже занятым номером получают ошибку. Семафоры команд и событий обозначают наличие сообщения; отдельные семафоры отмечают остановку, отмену и выход участников.

Завершение и освобождение ресурсов

Штатный выход происходит после подключения ожидаемых участников и подтверждения всех результатов и отказов. Если достигнут лимит времени, выводятся состояния оставшихся работ; в обоих случаях код выхода 0, а в журнале причины различаются. Ноль студентов или ноль билетов обрабатываются без ожидания несуществующих проверок.

`SIGINT` или `SIGTERM` у координатора устанавливает флаг, после чего тот публикует остановку и записывает итог. Обработчик сигнала не вызывает функции ввода-вывода. Сигнал независимому участнику передаёт координатору запрос отмены. Дочерние процессы забираются через `waitpid`; независимые участники закрывают собственные дескрипторы. Координатор закрывает и удаляет созданные IPC-объекты. Неименованные семафоры уничтожаются после выхода дочерних процессов. Коды выхода координатора: 130 для `SIGINT`, 143 для `SIGTERM`, 2 для ошибок.

Консольный интерфейс

Сборка и тесты выполняются в Linux или Ubuntu WSL. Нужны компилятор C, `make` и Python 3 для тестов. Для примеров из копии на диске D:

```
cd /mnt/d/codex/Prj17/hw3
make
./exam-unnamed --delay-ms 100
./exam-named --delay-ms 100 --log named.log
make test
```

Пример независимой сессии с одним студентом и одним преподавателем. Сначала запустить координатор. После строки «ГОТОВО» открыть участников в двух других терминалах из той же папки:

```
./exam-shm --session demo --students 1 --teachers 1 \
  --delay-ms 100 --max-time 0

./exam-teacher --mode shm --session demo --id 1

./exam-student --mode shm --session demo --id 1
```

Для очередей сообщений используется `exam-mq`, у участников указывается `--mode mq`. Скрипт автоматизирует запуск тех же отдельных программ и ждёт их завершения:

```
bash launch_scripts/independent.sh shm 6 2
bash launch_scripts/independent.sh mq 6 2
bash launch_scripts/showcase.sh
```

Параметр	Назначение и значение по умолчанию
`--students`, `--teachers`, `--tickets`	6 студентов, 2 преподавателя, 6 билетов
`--queue`, `--check-time`	3 места, проверка за 2 единицы
`--prep-min`, `--prep-max`	От 2 до 5 единиц на подготовку
`--delay-ms`, `--max-time`	10 мс на единицу; лимит 100 единиц
`--ticket-mode`, `--grade-mode`	`random` по умолчанию; также `sequential` и `ticket` соответственно
`--seed`, `--log`	17, журнал `exam.log`
`--session`	Обязательное имя при независимом запуске

`--delay-ms 0` означает 1 мс на единицу, `--max-time 0` снимает лимит. Для независимого запуска лимит включает подключение участников. Справка `--help` показывает допустимые диапазоны. Строки журнала отмечены реальными миллисекундами от начала. Координатор пишет одинаковые события в stdout и журнал через файловые дескрипторы. Итог содержит число допущенных, завершивших и ожидавших, максимум очереди и среднюю оценку.

Проверка и результаты

В Ubuntu 24.04 под WSL выполнена сборка C11 с `-Wall -Wextra -Wpedantic -Werror` и 48 сценариев. Ниже приведены четыре штатных запуска с 6 студентами, 2 преподавателями, 6 билетами и увеличенным временем проверки, чтобы было видно перекрытие работ.

Режим	Завершили	Пик очереди	Ожидали	Средняя оценка
Неименованные семафоры	6 из 6	3	1	3,17
Именованные семафоры	6 из 6	3	1	3,17
Независимый запуск, память	6 из 6	3	1	3,17
Независимый запуск, сообщения	6 из 6	3	1	3,17

Примеры журналов сохранены в `results/sample_*.log`. Файл `test-summary.txt` перечисляет выполненные сценарии.

- Проверялись уникальность билетов, FIFO, вместимость очереди, отсутствие повторных проверок, принадлежность оценки и баланс итогов.
- Очередь на одно место действительно заставляла студентов ждать. При двух билетах из шести студентов завершили экзамен двое, четверо получили отказ. При 32 студентах и 32 преподавателях завершили все 32.
- При ограничении времени, `SIGINT`, `SIGTERM` и аварийном выходе участника проверялись причина завершения, коды выхода и отсутствие объектов с именем сессии в `/dev/shm` и `/dev/mqueue`. Для дочерних процессов проверено отсутствие оставшихся PID.
- Для независимого запуска проверены отсутствие участника, повторное имя активной сессии и повторный номер преподавателя.

Использование ИИ

Chat GPT использовался для подготовки автоматических тестов. Формулировки запросов приведены в `README_AI.md`.

Тесты проверяют штатное завершение, переполнение очереди, остановку по сигналам и удаление объектов IPC после выхода. Проверки опираются на события в журнале и итоговые состояния: точное время и порядок событий могут меняться от запуска к запуску.